Delivery Semantics

at-most-once, at-least-once, exactly-once

Watch the 4-min video

In one line: whether a consumer commits its position before or after processing decides between losing messages (at-most-once) and repeating them (at-least-once); most systems choose at-least-once with idempotent handlers, and get exactly-once effects by committing results and offsets atomically.

1 One decision

A consumer reads a message, writes the result to the database, and crashes. On restart, does it read that message again, or skip it? The answer depends on one decision: **when** the consumer records its progress (commits the offset or acknowledges the message) relative to doing the work.



Figure 1. Commit before processing loses messages on a crash; commit after processing repeats them.

2 The three semantics

At-most-once: commit first, then process. A crash loses the message, but it's never handled twice. **At-least-once:** process first, then commit. A crash means the message is processed again. **Exactly-once:** the effect happens once. You get it by making processing idempotent, or by committing the result and the offset in one atomic step (lesson 21). The producer side has the same choice: no retries (at-most-once) or retries (at-least-once, made safe by an idempotent producer).

WORKED EXAMPLE: ORDERS INTO POSTGRES

A consumer reads order events and writes them into Postgres. It disables auto-commit. For each batch, it upserts the orders by `order_id` in one database transaction, then commits the Kafka offsets. If it crashes after the database commit but before the offset commit, the batch is re-read on restart, and the upserts change nothing: at-least-once delivery with an idempotent sink, so exactly-once effect. A stricter variant stores the last processed offset per partition in the same Postgres transaction and seeks to it on startup.

```
consumer = KafkaConsumer('orders', enable_auto_commit=False, ...)
for batch in consumer:
    with db.transaction():
        for r in batch:
            db.execute('INSERT ... ON CONFLICT (order_id) DO UPDATE ...', r)
    consumer.commit()        # after the DB commit
```

Figure 2. At-least-once with an idempotent upsert: duplicates are harmless.

3 Which to choose

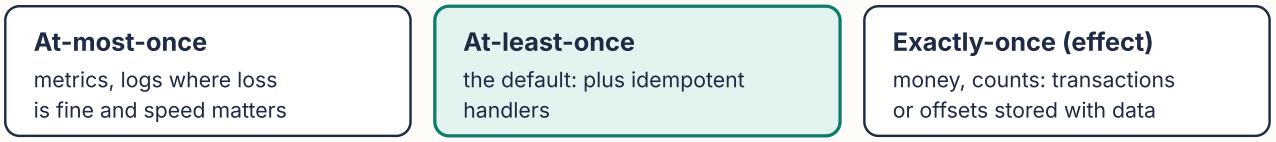


Figure 3. Choose per use case. At-least-once plus idempotency covers most needs.

Auto-commit in Kafka clients commits on a timer in the background, so it can commit offsets for messages your code hasn't finished: effectively at-most-once for those, at-least-once for others. Prefer explicit commits for anything important. In queues like SQS or RabbitMQ, the same rule applies to when you **delete or ack** the message.

COMMON MISTAKES

- Auto-commit with asynchronous processing that may fail later.
- Acking a message before handing it to a background task.
- At-least-once without idempotent handlers.
- Committing the offset of the last record when earlier ones in the batch failed.

4 Key terms

At-most-once: possibly lost, never duplicated

At-least-once: never lost, possibly duplicated

Exactly-once effect: duplicates delivered but applied once

Offset commit / ack: recording that a message is done

Idempotent sink: writing the same record twice changes nothing

5 Interview questions

Q1. What decides at-most-once vs at-least-once on the consumer? BASIC
Whether the offset or ack is committed before processing (at-most-once) or after (at-least-once).

Q2. Why is at-least-once the usual default? BASIC
Losing data is usually worse than processing it twice, and duplicates can be made harmless with idempotent handlers.

Q3. Design a consumer that writes events to a database without duplicates or losses. SCENARIO
Disable auto-commit, upsert by a unique event or entity key inside a DB transaction, then commit offsets; or store offsets in the same DB transaction.

Q4. Metrics events at 1 million per second: which semantics? SCENARIO
At-most-once or at-least-once with auto-commit is fine; losing or double-counting a few metrics is acceptable, and speed matters more.

Q5. Can auto-commit lose messages? TRICKY
Yes: it may commit offsets for records that were fetched but not yet processed; a crash then skips them.

Q6. Where can duplicates come from even with careful commits? TRICKY

Producer retries without idempotence, rebalances before a commit, redelivery after a visibility timeout, and replays.

Next: SD-I-009 · Dead-Letter Queues